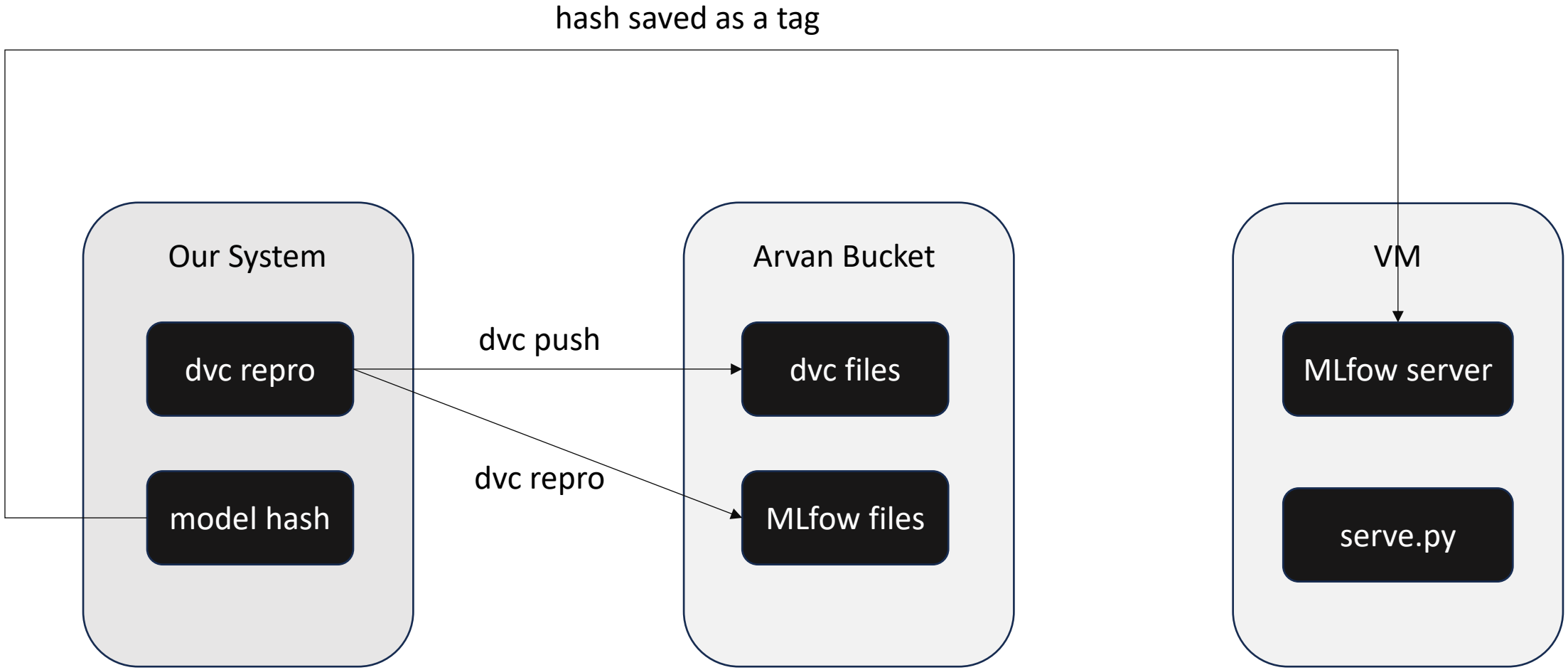
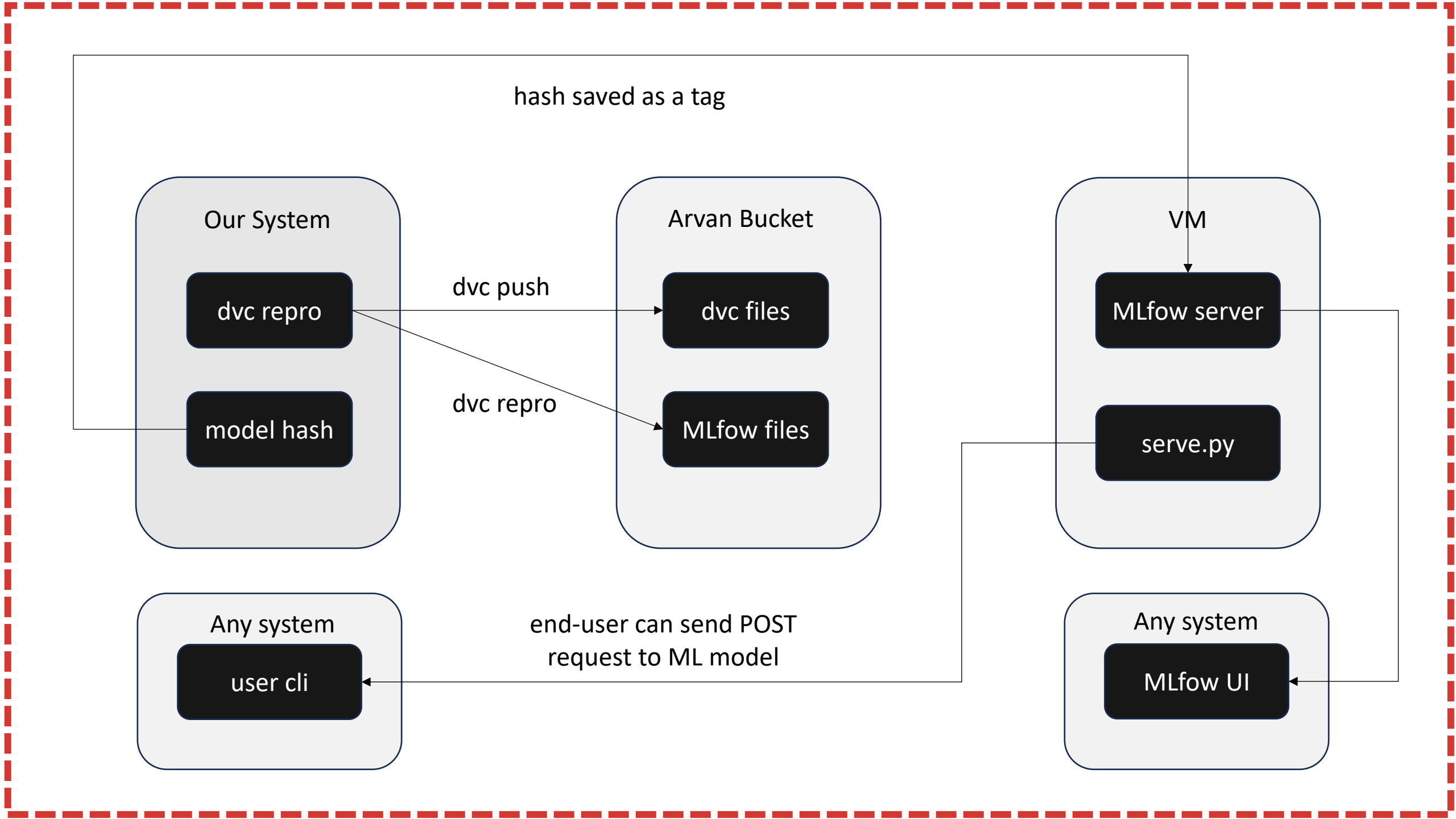
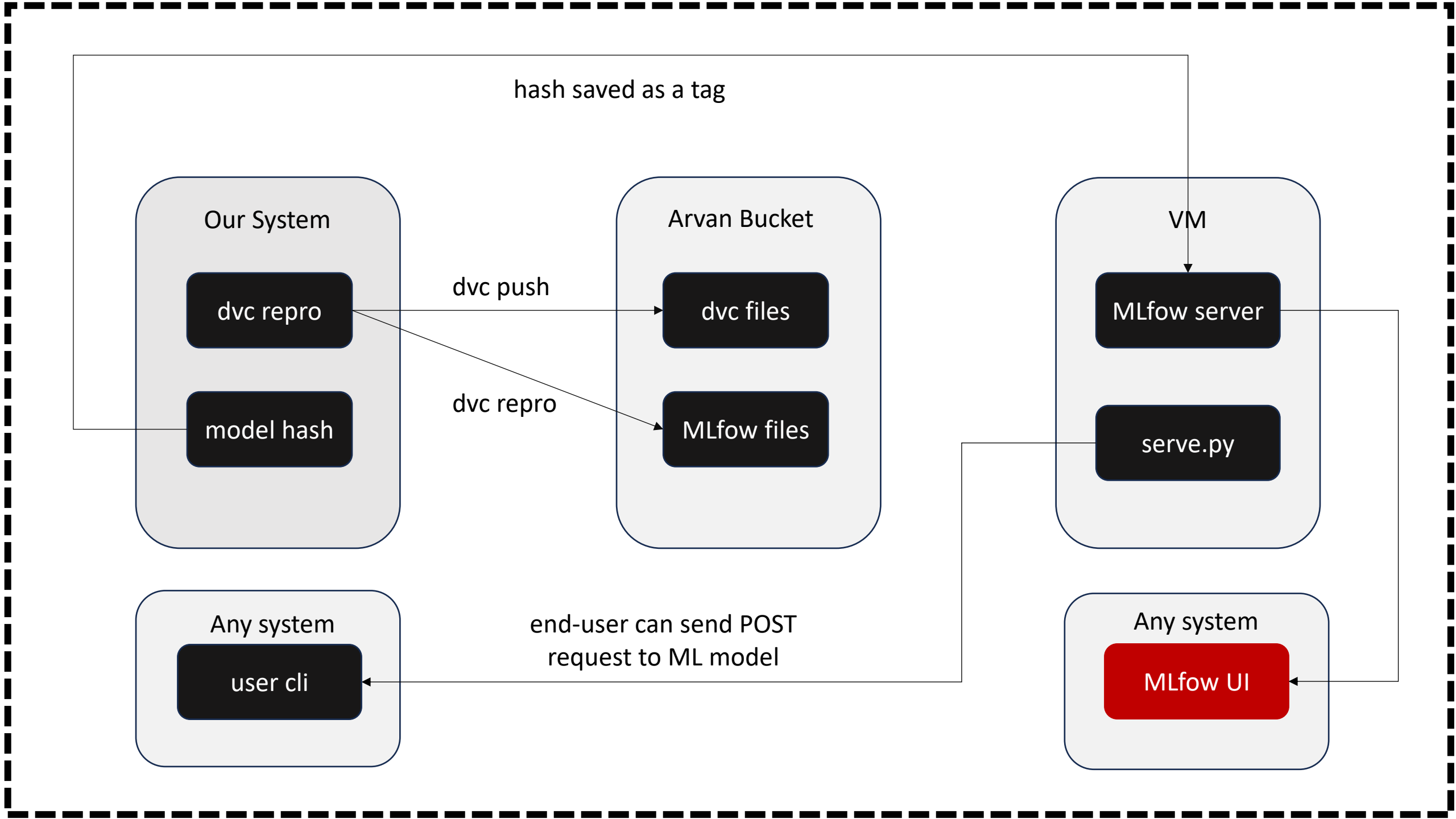
Security issues in Mlflow Model Hash, Auth and Systemd

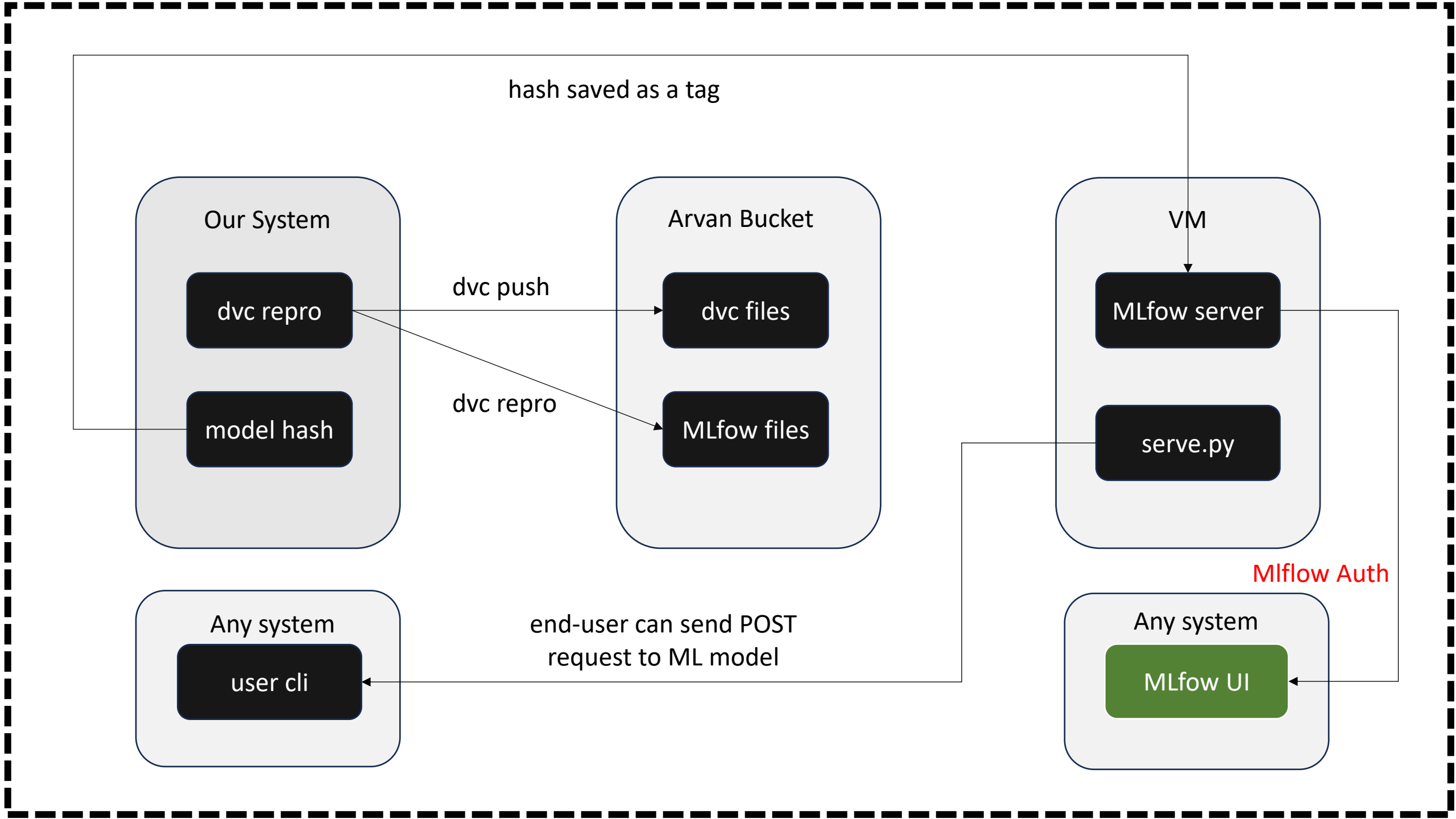
Mohammad Fozouni (Ph.D.)

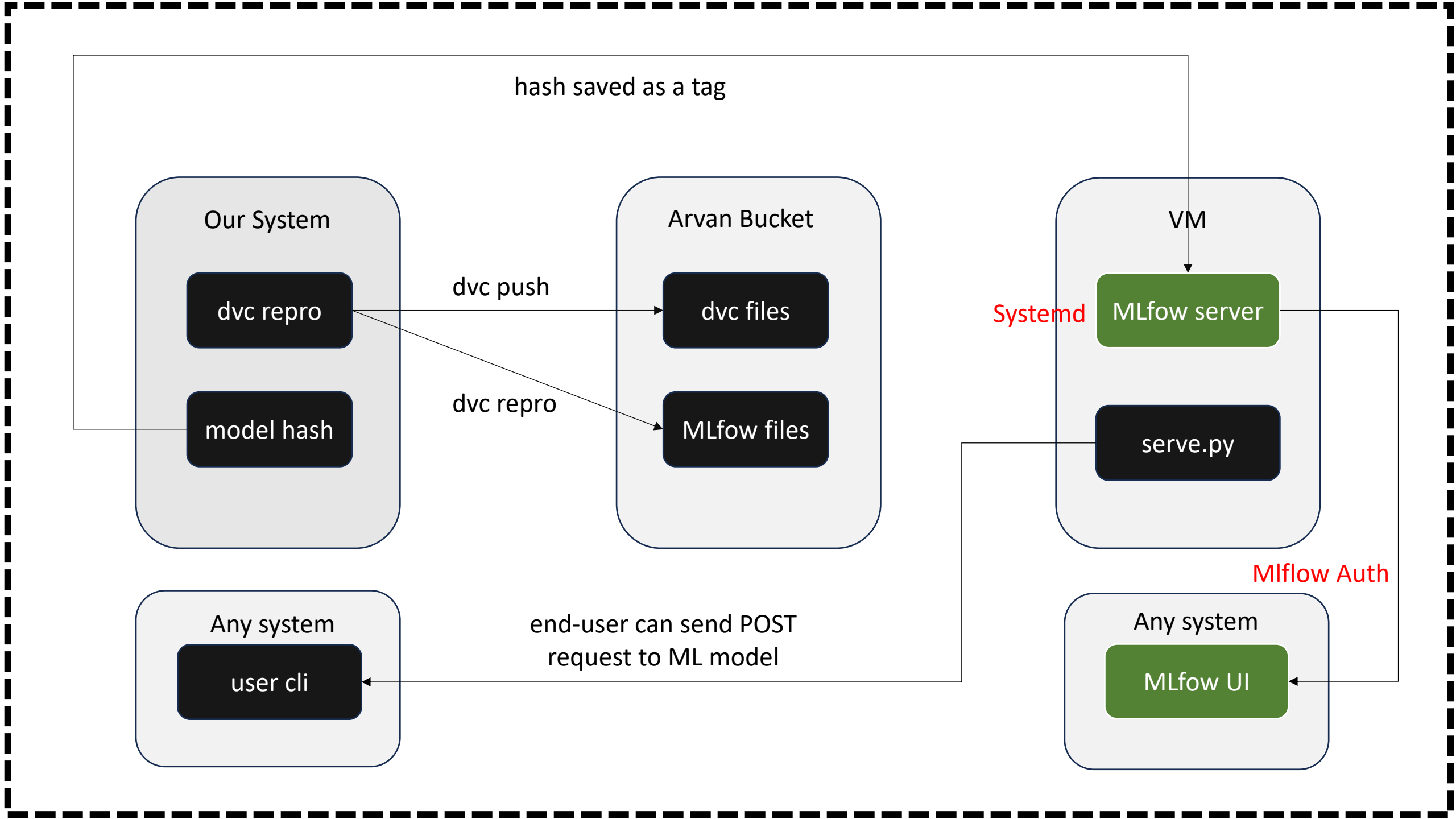
MLSECOPS IN ACTION COURSE


hash saved as a tag
Our System
dvc repro
model hash
dvc push
dvc repro
Arvan Bucket
dvc files
MLfow files
VM
MLfow server
serve.py


hash saved as a tag
Our System
dvc repro
model hash
dvc push
dvc repro
Arvan Bucket
dvc files
MLfow files
VM
MLfow server
serve.py
Any system
user cli
end-user can send POST
request to ML model
Any system
MLfow UI

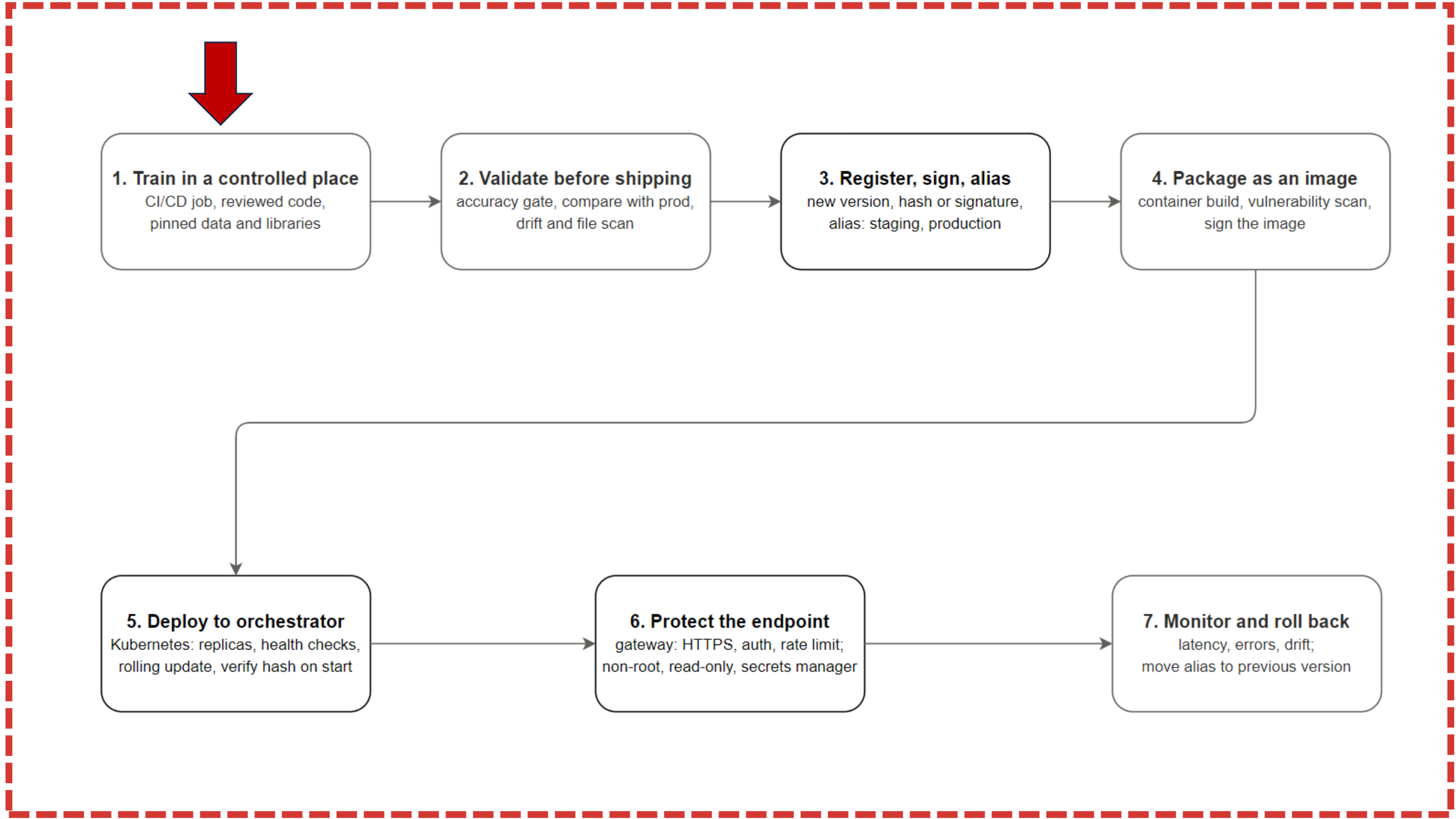

hash saved as a tag
Our System
dvc repro
model hash
dvc push
dvc repro
Arvan Bucket
dvc files
MLfow files
VM
MLfow server
serve.py
Any system
user cli
end-user can send POST
request to ML model
Any system
MLfow UI

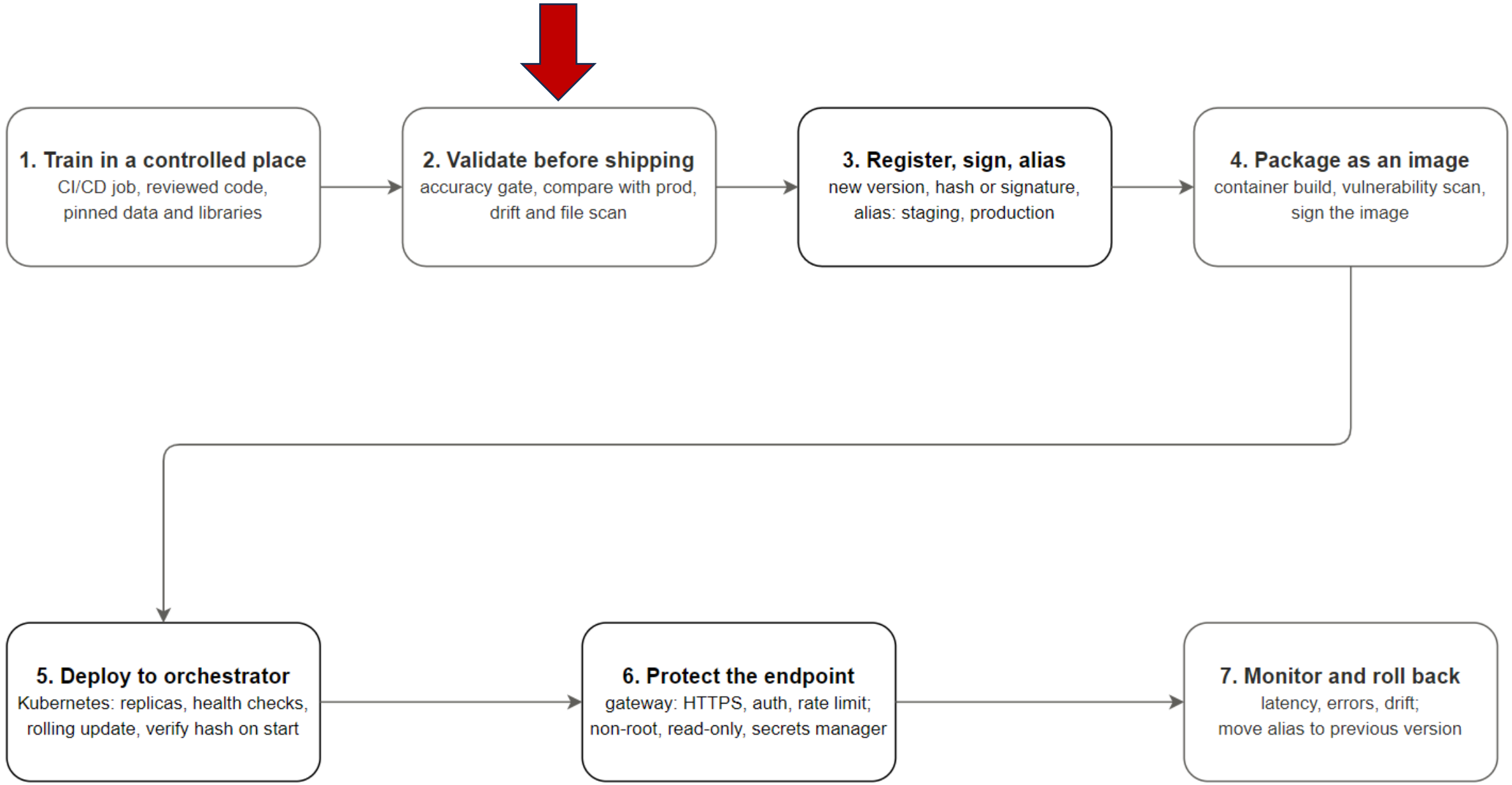

hash saved as a tag
Our System
dvc repro
model hash
dvc push
dvc repro
Arvan Bucket
dvc files
MLfow files
VM
MLfow server
serve.py
Mlflow Auth
Any system
user cli
end-user can send POST
request to ML model
Any system
MLfow UI

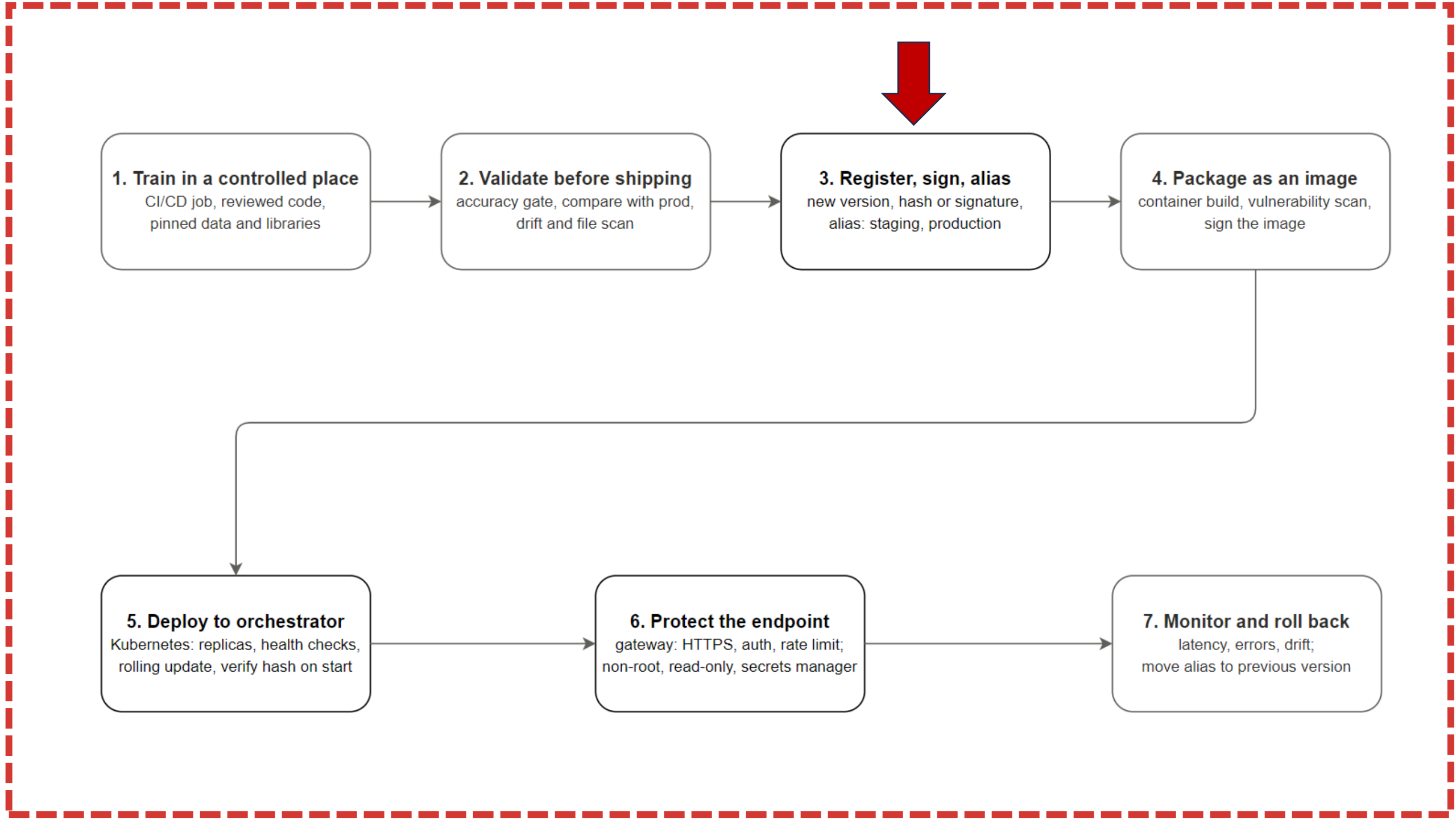

hash saved as a tag
Our System
dvc repro
model hash
dvc push
dvc repro
Arvan Bucket
dvc files
MLfow files
VM
Systemd
MLfow server
serve.py
Mlflow Auth
Any system
user cli
end-user can send POST
request to ML model
Any system
MLfow UI

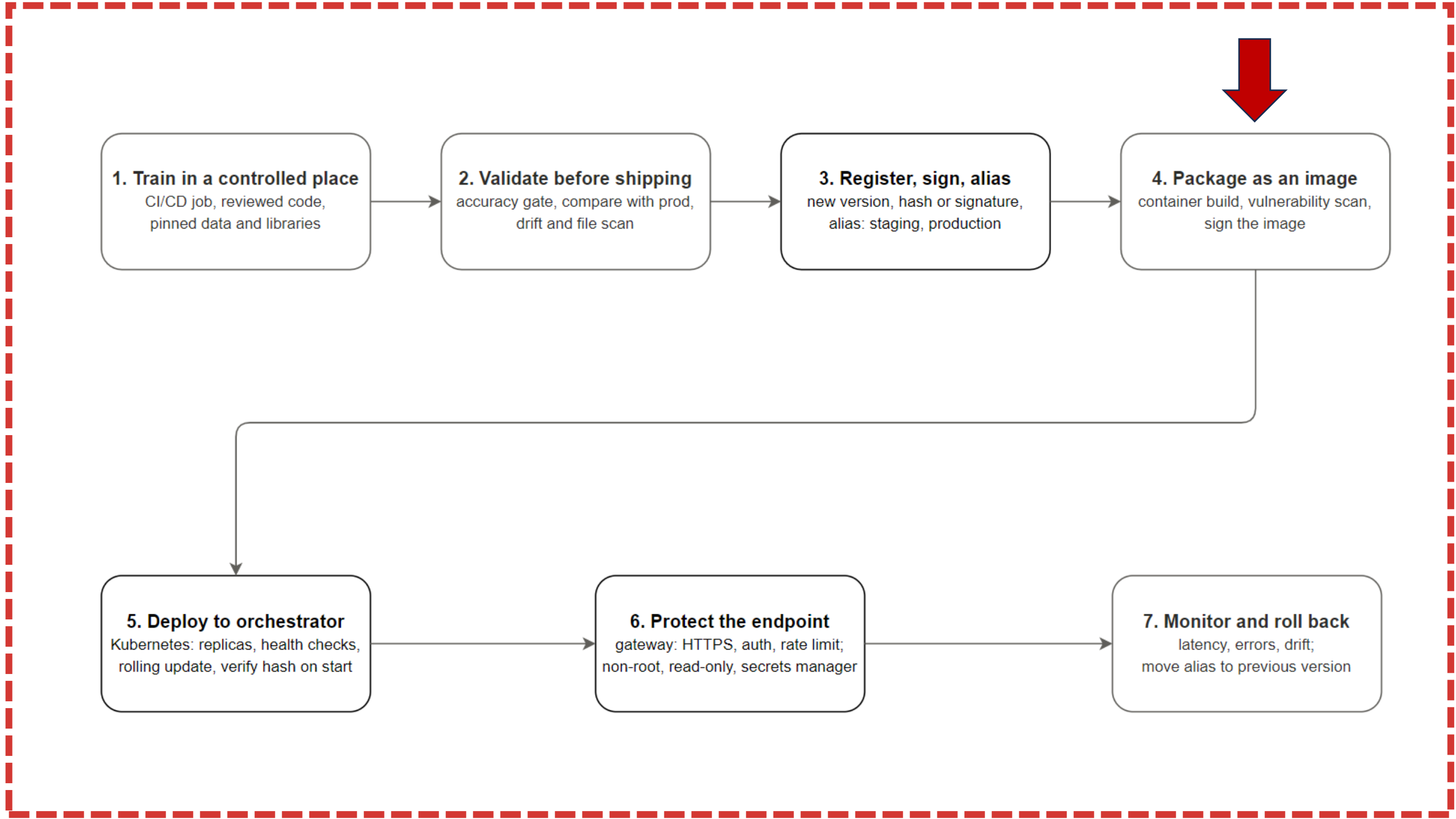
How ML models go to prod in big corporations?

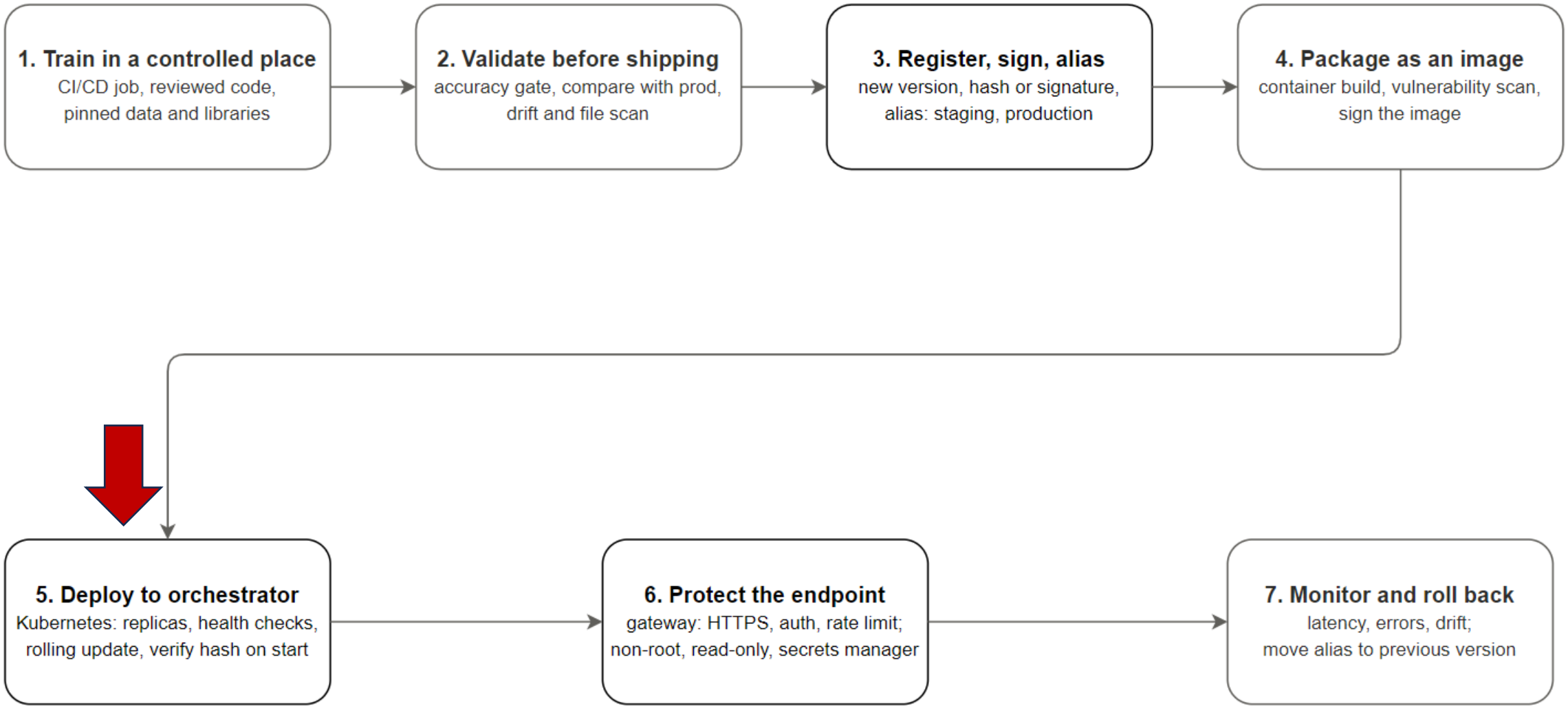


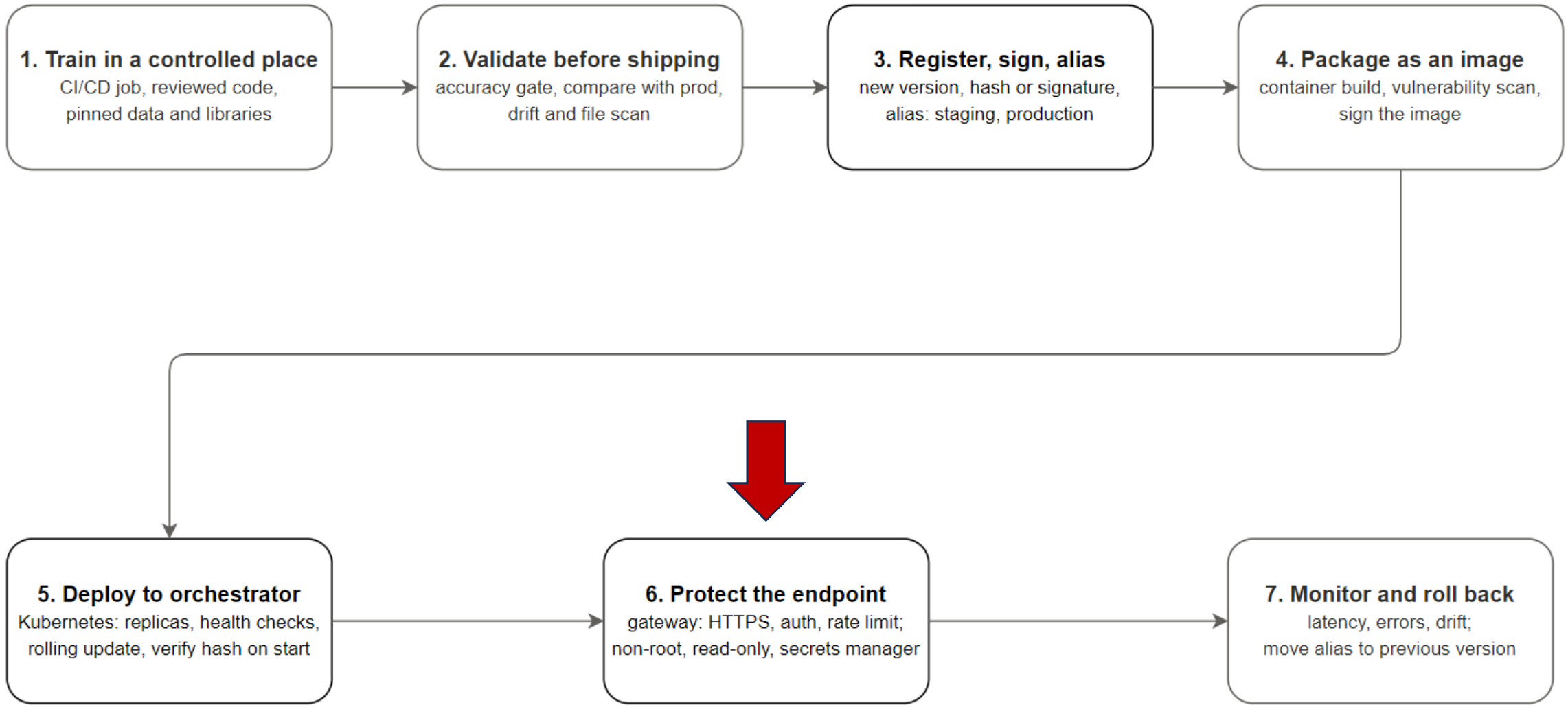

1. Train in a controlled place
CI/CD job, reviewed code, pinned data and libraries
2. Validate before shipping
accuracy gate, compare with prod, drift and file scan
3. Register, sign, alias
new version, hash or signature, alias: staging, production
4. Package as an image
container build, vulnerability scan, sign the image
5. Deploy to orchestrator
Kubernetes: replicas, health checks, rolling update, verify hash on start
6. Protect the endpoint
gateway: HTTPS, auth, rate limit; non-root, read-only, secrets manager
7. Monitor and roll back
latency, errors, drift; move alias to previous version

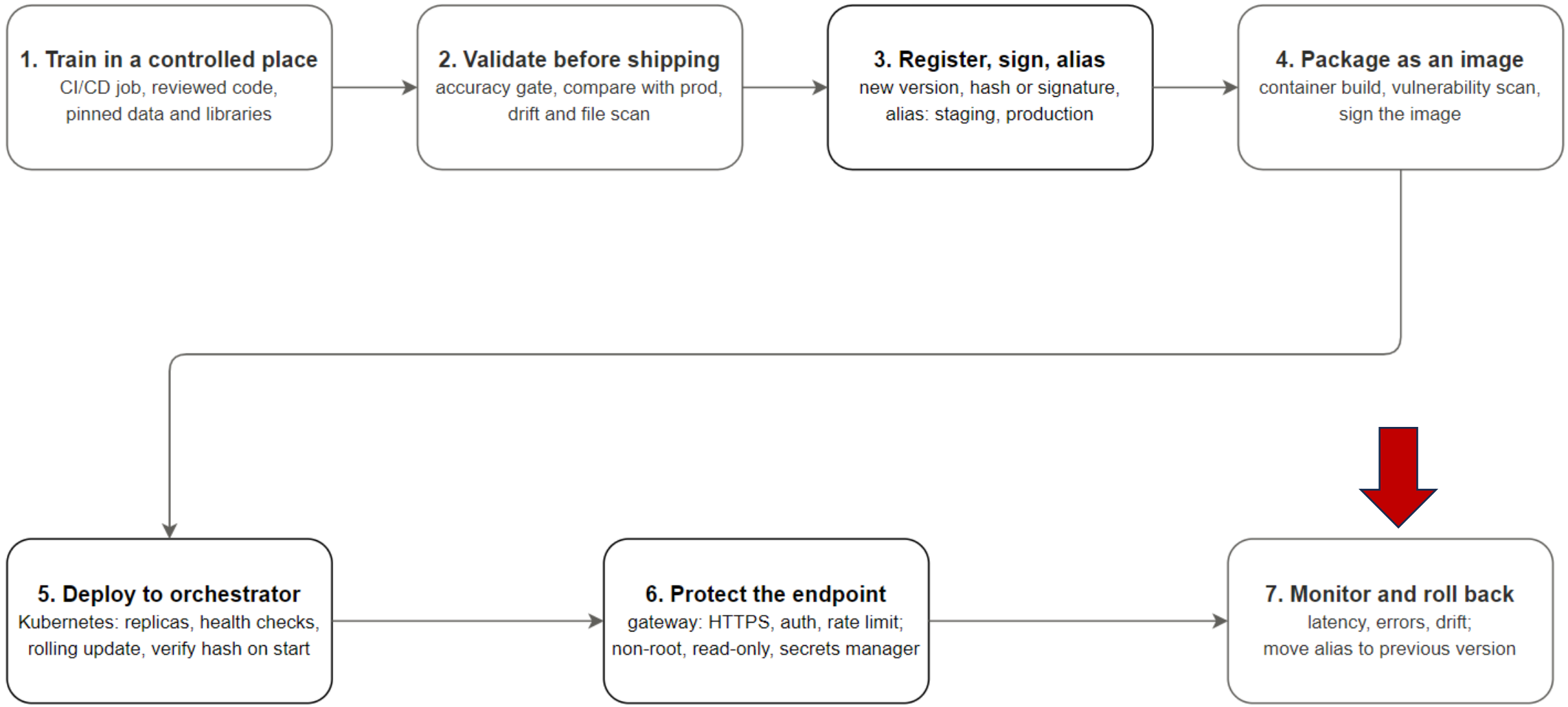

1. Train in a controlled place
CI/CD job, reviewed code, pinned data and libraries
2. Validate before shipping
accuracy gate, compare with prod, drift and file scan
3. Register, sign, alias
new version, hash or signature, alias: staging, production
4. Package as an image
container build, vulnerability scan, sign the image
5. Deploy to orchestrator
Kubernetes: replicas, health checks, rolling update, verify hash on start
6. Protect the endpoint
gateway: HTTPS, auth, rate limit; non-root, read-only, secrets manager
7. Monitor and roll back
latency, errors, drift; move alias to previous version

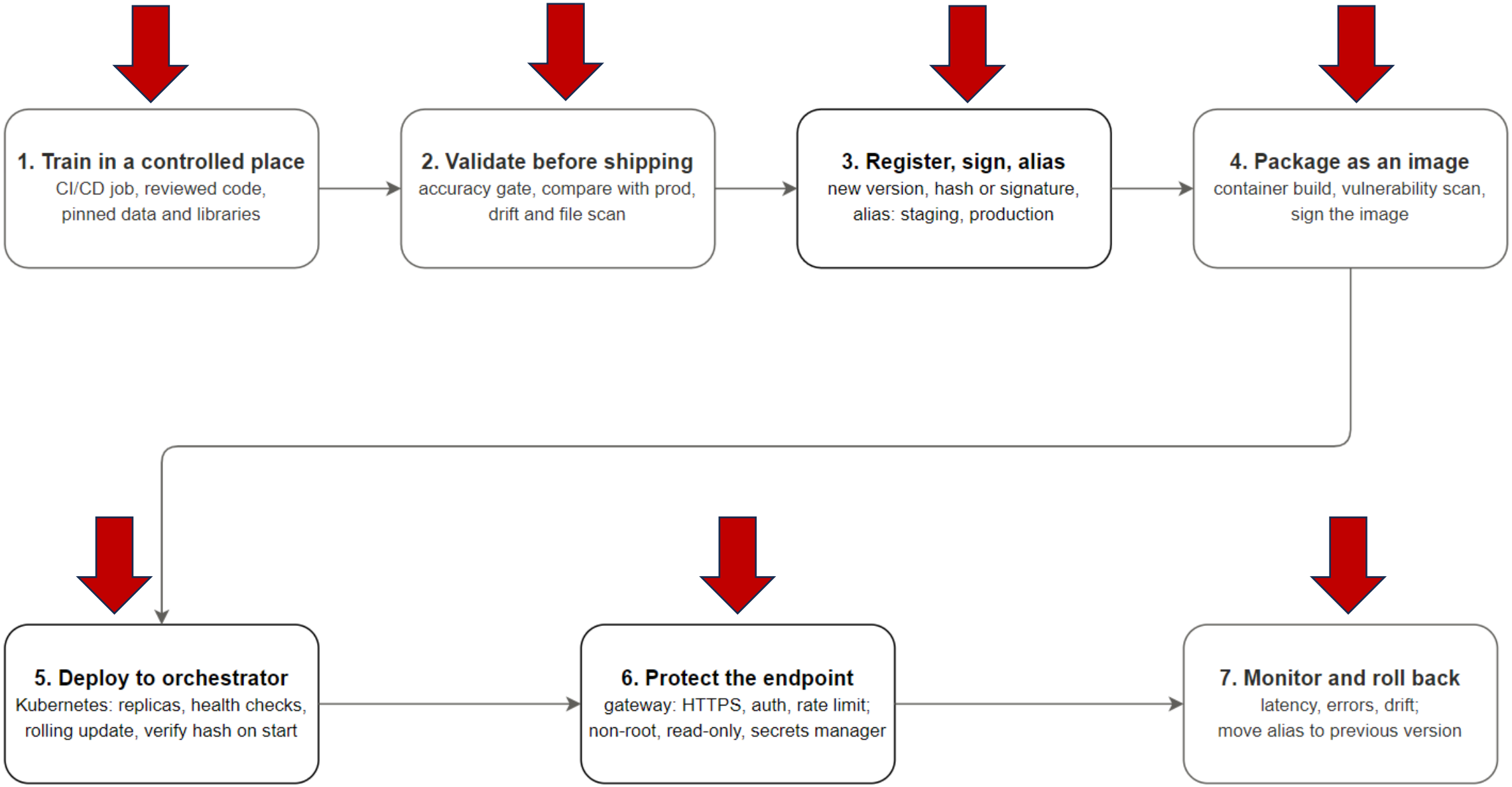

1. Train in a controlled place
CI/CD job, reviewed code, pinned data and libraries
2. Validate before shipping
accuracy gate, compare with prod, drift and file scan
3. Register, sign, alias
new version, hash or signature, alias: staging, production
4. Package as an image
container build, vulnerability scan, sign the image
5. Deploy to orchestrator
Kubernetes: replicas, health checks, rolling update, verify hash on start
6. Protect the endpoint
gateway: HTTPS, auth, rate limit; non-root, read-only, secrets manager
7. Monitor and roll back
latency, errors, drift; move alias to previous version


1. Train in a controlled place
CI/CD job, reviewed code, pinned data and libraries
2. Validate before shipping
accuracy gate, compare with prod, drift and file scan
3. Register, sign, alias
new version, hash or signature, alias: staging, production
4. Package as an image
container build, vulnerability scan, sign the image
5. Deploy to orchestrator
Kubernetes: replicas, health checks, rolling update, verify hash on start
6. Protect the endpoint
gateway: HTTPS, auth, rate limit; non-root, read-only, secrets manager
7. Monitor and roll back
latency, errors, drift; move alias to previous version


1. Train in a controlled place
CI/CD job, reviewed code, pinned data and libraries
2. Validate before shipping
accuracy gate, compare with prod, drift and file scan
3. Register, sign, alias
new version, hash or signature, alias: staging, production
4. Package as an image
container build, vulnerability scan, sign the image
5. Deploy to orchestrator
Kubernetes: replicas, health checks, rolling update, verify hash on start
6. Protect the endpoint
gateway: HTTPS, auth, rate limit; non-root, read-only, secrets manager
7. Monitor and roll back
latency, errors, drift; move alias to previous version


1. Train in a controlled place
CI/CD job, reviewed code, pinned data and libraries
2. Validate before shipping
accuracy gate, compare with prod, drift and file scan
3. Register, sign, alias
new version, hash or signature, alias: staging, production
4. Package as an image
container build, vulnerability scan, sign the image
5. Deploy to orchestrator
Kubernetes: replicas, health checks, rolling update, verify hash on start
6. Protect the endpoint
gateway: HTTPS, auth, rate limit; non-root, read-only, secrets manager
7. Monitor and roll back
latency, errors, drift; move alias to previous version


1. Train in a controlled place
CI/CD job, reviewed code, pinned data and libraries
2. Validate before shipping
accuracy gate, compare with prod, drift and file scan
3. Register, sign, alias
new version, hash or signature, alias: staging, production
4. Package as an image
container build, vulnerability scan, sign the image
5. Deploy to orchestrator
Kubernetes: replicas, health checks, rolling update, verify hash on start
6. Protect the endpoint
gateway: HTTPS, auth, rate limit; non-root, read-only, secrets manager
7. Monitor and roll back
latency, errors, drift; move alias to previous version


1. Train in a controlled place
CI/CD job, reviewed code, pinned data and libraries
2. Validate before shipping
accuracy gate, compare with prod, drift and file scan
3. Register, sign, alias
new version, hash or signature, alias: staging, production
4. Package as an image
container build, vulnerability scan, sign the image
5. Deploy to orchestrator
Kubernetes: replicas, health checks, rolling update, verify hash on start
6. Protect the endpoint
gateway: HTTPS, auth, rate limit; non-root, read-only, secrets manager
7. Monitor and roll back
latency, errors, drift; move alias to previous version

Canary deployment of ML models

Using Traefik you can run two versions of your model API side-by-side and use Traefik's **weighted load balancing** to send, say, 99% of traffic to your stable model and 1% to your new "canary" model. You can then monitor performance and gradually shift traffic if the new model performs well . This is a best practice for safely testing ML models in production.



Canary deployment of ML models

Using Traefik you can run two versions of your model API side-by-side and use Traefik's **weighted load balancing** to send, say, 99% of traffic to your stable model and 1% to your new "canary" model. You can then monitor performance and gradually shift traffic if the new model performs well . This is a best practice for safely testing ML models in production.



A critical consideration

If MLflow runs as root and hackers find a hole in it, they get root on the whole machine. If MLflow runs as a low-privilege user, the same hole gives them only what that user can access, which is much less.

The attacker is still inside, but they can only touch what that account can touch, so the damage is smaller.



A critical consideration

If MLflow runs as root and hackers find a hole in it, they get root on the whole machine. If MLflow runs as a low-privilege user, the same hole gives them only what that user can access, which is much less.

The attacker is still inside, but they can only touch what that account can touch, so the damage is smaller.

پس تا می‌تونید، روی سرور با کاربر root هیچ مولفه و پراسسی را بالا نیارید. برای هر کاری، یک کاربر مخصوص باید بسازیم. دسترسی‌هاش رو کم کنیم و از chmod 600 هم غافل نشیم.



A critical consideration

If MLflow runs as root and hackers find a hole in it, they get root on the whole machine. If MLflow runs as a low-privilege user, the same hole gives them only what that user can access, which is much less.

The attacker is still inside, but they can only touch what that account can touch, so the damage is smaller.

پس تا می‌تونید، روی سرور با کاربر root هیچ مولفه و پراسسی را بالا نیارید. برای هر کاری، یک کاربر مخصوص باید بسازیم. دسترسی‌هاش رو کم کنیم و از chmod 600 هم غافل نشیم.

اما نترسید. در ساده‌ترین حالت ممکن سرویس رو بیارید بالا و تلاش کنید که به مرور زمان امنش کنید.





Now Let’s go for practical stuffs